✦ NAKSHATRA

Agent Todo — Build Checklist

31 tasks across 10 milestones

Vite · React · TypeScript · Three.js · Zustand · astronomy-engine · luxon

■ Milestone 1 — Project Foundation

T00 Project Setup

Scaffold the Nakshatra project with Vite, React, TypeScript, Three.js, Zustand, Tailwind CSS, GSAP, astronomy-engine, luxon, and geo-tz. Create the full folder structure as specified in the PRD.

Acceptance Criteria

- ✓ Project runs locally with no TypeScript errors
- ✓ Three.js is installed and a blank canvas renders
- ✓ All folders exist: components/, three/, services/, state/, types/
- ✓ README added with project purpose and setup commands

T01 Main Visual Layout Shell

Build the full-screen dark cosmic UI shell matching the concept image. Left glassmorphism panel, center canvas area, right location card, bottom timeline, bottom-right control dock, top navigation.

Acceptance Criteria

- ✓ Layout matches concept image direction
- ✓ Left panel uses glass-style UI with backdrop blur
- ✓ Background is dark and cinematic (#030712)
- ✓ Top nav shows logo, links, My Snapshots, profile icon
- ✓ All placeholder areas are correctly positioned

■ Milestone 2 — 3D Scene Foundation

T02 Three.js Canvas & Scene

Implement the Three.js scene inside the main canvas area. Scene, PerspectiveCamera, WebGLRenderer, resize handling, animation loop, basic lighting. Camera at position (0, 45, 75) looking at origin.

Acceptance Criteria

- ✓ Three.js canvas renders inside the page
- ✓ Canvas resizes with the browser window
- ✓ A placeholder glowing sphere appears at center
- ✓ No memory leaks on component unmount
- ✓ View feels semi-top-down, not first-person

T03 Starfield Background

Create a reusable Starfield module using Three.js instanced particles. Stars fill the scene background with subtle variation in size and brightness. Add subtle twinkling if performance allows.

Acceptance Criteria

- ✓ Scene has a rich deep-space background
- ✓ Small white/blue dots with a few brighter stars
- ✓ Performance remains smooth (60 FPS)
- ✓ Starfield sits behind the solar system visually

T04 Static Stylized Solar System

Create a static 2.5D solar system with Sun, Mercury, Venus, Earth, Mars, Jupiter, Saturn (Uranus/Neptune optional). Use placeholder orbit positions. Add elliptical orbit rings on XZ plane. Style each planet distinctly.

Acceptance Criteria

- ✓ Beautiful static solar system is visible
- ✓ Sun is orange/gold and feels central
- ✓ Orbit rings are thin with blue/gold glow
- ✓ Saturn has a visible ring
- ✓ Each planet is clearly distinguishable

■ Milestone 3 — Camera & Interaction

T05 Camera Orbit Controls

Add orbit/zoom camera controls. Left drag to orbit, scroll to zoom, right drag to pan. Add zoom limits, camera flip prevention, and a reset function. Disable controls during rewind animation.

Acceptance Criteria

- ✓ Mouse drag orbits the camera smoothly
- ✓ Scroll zooms in/out without going inside planets
- ✓ Camera does not flip upside down
- ✓ Reset camera returns to default position
- ✓ Controls feel premium, not jittery

T06 Floating Planet Labels

Add planet name labels near each planet using CSS2DRenderer or HTML overlay. Labels face the camera at all times. Implement toggle on/off via the control dock.

Acceptance Criteria

- ✓ All 6+ planets have visible, readable labels
- ✓ Labels move correctly with planet positions
- ✓ Labels can be hidden via UI toggle
- ✓ Scene still looks clean with labels on

T07 Compass Rose

Add an N/S/E/W compass rose to the top-right corner of the 3D scene. The compass should visually reflect the camera's current orientation as the user orbits.

Acceptance Criteria

- ✓ Compass rose is visible in top-right of scene
- ✓ N/S/E/W labels are readable
- ✓ Compass updates correctly when camera orbits

■ Milestone 4 — Form, Validation & State

T08 Birth Form With Validation

Implement the birth input form with date picker, time picker, latitude, longitude, and timezone fields. Auto-infer timezone from lat/long using geo-tz when both are entered. Show friendly inline validation errors.

Acceptance Criteria

- ✓ User can enter all birth details
- ✓ Timezone auto-infers from lat/long on blur
- ✓ Inline errors appear for invalid latitude/longitude
- ✓ Valid submit calls onBirthSubmit(input)
- ✓ Form looks premium, not like a default HTML form

T09 App State Modes

Create the Zustand store with AppMode ('live' | 'input' | 'rewinding' | 'birth-snapshot' | 'error') and full NakshatraState. Wire mode transitions to form submit, animation completion, and return-to-live action.

Acceptance Criteria

- ✓ App starts in live mode
- ✓ Submit transitions to rewinding
- ✓ Animation complete transitions to birth-snapshot
- ✓ Return to live clears snapshot state
- ✓ Mode is readable in debug/console

T10 Status Bar Component

Build the top-center two-state status bar matching the concept image: LIVE (green dot, arrow) and BIRTH SNAPSHOT (blue pause icon). The bar is a visual mode switcher, not just a pill.

Acceptance Criteria

- ✓ Status bar shows both states with arrow between them
- ✓ Active state is highlighted; inactive is muted
- ✓ Updates correctly on mode change
- ✓ Matches concept image layout

■ Milestone 5 — Astronomy Service

T11 Time Conversion Service

Create timeService.ts using luxon. Convert birth date + time + IANA timezone into a stable UTC timestamp. Do not use the browser's local timezone. Input and output types per PRD data models.

Acceptance Criteria

- ✓ Same input always returns same UTC timestamp
- ✓ Different timezone changes UTC correctly
- ✓ Function is isolated from UI
- ✓ Unit tests or console test examples provided

T12 Astronomy Service Interface

Create the AstronomyProvider abstraction with getSolarSystemSnapshot(timestampUtc). First implement with simplified circular-orbit math so positions vary by date. Wire to scene for visual testing.

Acceptance Criteria

- ✓ Function returns correct PlanetPosition[] structure
- ✓ Different dates return different positions
- ✓ Scene consumes snapshot data correctly
- ✓ Interface is clean and swappable

T13 Real Astronomy Engine (astronomy-engine)

Replace fake orbit math with astronomy-engine calculations for Mercury, Venus, Earth, Mars, Jupiter, Saturn. Convert ecliptic coordinates to Three.js XZ plane. Use artistic scaling — keep real angular relationships but compress distances.

Acceptance Criteria

- ✓ Planet positions are based on real timestamp
- ✓ Birth date produces visibly different arrangement from today
- ✓ Same date always produces same result
- ✓ Scene components do not know which provider is used

T14 Planet Position Scaling System

Create an isolated mapAstronomyPositionToScene() function with non-linear compression so all planets are visible in the same camera view. Add clear comments that scaling is artistic. Inner planets near Sun, outer planets visible but not off-screen.

Acceptance Criteria

- ✓ All planets visible in default camera view
- ✓ Inner planets do not overlap the Sun
- ✓ Jupiter and Saturn are clearly farther out
- ✓ Scaling logic is in one isolated function

■ Milestone 6 — Live Mode

T15 Live Mode — Real-Time Solar System

On page load, use the current UTC timestamp to compute and render planet positions. Update positions periodically (every 30–60 seconds or via smooth interpolation). Earth rotates slowly. Status bar shows LIVE.

Acceptance Criteria

- ✓ App opens with current solar system
- ✓ Planets are not frozen in live mode
- ✓ Current date/time is visible in the UI
- ✓ Status bar shows LIVE state
- ✓ Earth rotates slowly

■ Milestone 7 — Rewind Animation

T16 Timeline Scrubber UI

Build the bottom timeline component. Shows NOW (date/time) on left, BIRTH MOMENT (date/time, gold) on right. Center shows year markers (e.g. 2020, 2010, 2000, 1992, 1980). A glowing particle dot scrubs left during rewind. Play/pause button.

Acceptance Criteria

- ✓ Timeline shows NOW, year markers, and BIRTH MOMENT
- ✓ Particle dot animates from right to left during rewind
- ✓ Pauses and shows final state in birth-snapshot mode
- ✓ Matches concept image layout exactly

T17 Rewind Animation — Core

On valid submit: get current and birth snapshots, set mode to rewinding, run 4-second GSAP ease-in-out interpolation of planet XZ positions. Animate timeline progress. On complete: set mode to birth-snapshot, freeze planets, enable camera.

Acceptance Criteria

- ✓ Planets visibly move from current to birth positions
- ✓ Animation takes 4 seconds with smooth easing
- ✓ Timeline scrubs correctly during animation
- ✓ App pauses cleanly at birth moment
- ✓ No abrupt jumps

T18 Reduced Motion Mode

Add a reduced-motion toggle (also respect prefers-reduced-motion). When enabled: skip the 4-second animation, fade directly from live to birth snapshot. All UI transitions become minimal.

Acceptance Criteria

- ✓ Toggle is accessible from settings or control dock
- ✓ Rewind animation is skipped when enabled
- ✓ App transitions directly to birth-snapshot mode
- ✓ App remains fully usable and beautiful

■ Milestone 8 — Earth Marker & Location

T19 EarthWithMarker Component

Create EarthWithMarker in Three.js. Render Earth sphere with atmosphere glow. Accept lat/long, convert to sphere coordinates using the PRD formula, place a glowing blue pin mesh parented to Earth. Orient Earth so marker is visible after rewind.

Acceptance Criteria

- ✓ Different lat/long values move the marker correctly
- ✓ Marker sits on Earth surface, not floating
- ✓ Marker glows and is clearly visible
- ✓ Earth orients toward marker after snapshot

T20 Geocode Service — Place Name

Create geocodeService.ts using OpenStreetMap Nominatim reverse geocoding. Given latitude and longitude, return a human-readable place name (e.g. 'Bengaluru, India'). Show in the location card. Handle failures gracefully.

Acceptance Criteria

- ✓ Place name appears in location card after submit
- ✓ Matches concept image (e.g. 'Bengaluru, India')
- ✓ Failure falls back to showing coordinates only
- ✓ API call is debounced or called once on submit

T21 Location Card Component

Build the right-side Location Card showing: place name, mini Earth globe with pin, latitude, longitude, timezone, birth date/time. 'View on Map' button links to Google Maps with coordinates. Uses glassmorphism style.

Acceptance Criteria

- ✓ Card appears after birth input is submitted
- ✓ All values match user input
- ✓ Mini globe with pin is visible
- ✓ View on Map opens Google Maps correctly
- ✓ Card does not obscure main solar system

■ Milestone 9 — Polish & Controls

T22 Control Dock

Build bottom-right control dock with icon buttons: Orbits, Planets, Stars, Labels, Settings. Active buttons glow blue. Also wire: Reset Camera and Return to Live. Match the concept image icon style.

Acceptance Criteria

- ✓ All toggles work correctly
- ✓ Active state has blue glow
- ✓ Reset camera restores default view
- ✓ Return to live clears snapshot and resumes live mode
- ✓ Dock matches concept image

T23 Birth Snapshot Success State

When rewind completes, show success message in left panel: 'System rewound to your birth moment. Paused and ready for exploration.' Status bar switches to BIRTH SNAPSHOT. Planets freeze. Earth marker remains. Camera controls activate.

Acceptance Criteria

- ✓ Success message is readable and satisfying
- ✓ Planets stop moving completely
- ✓ Status bar is in BIRTH SNAPSHOT state
- ✓ Camera is freely explorable

T24 Sun Glow, Bloom & Lighting

Add Three.js EffectComposer with UnrealBloomPass for bloom. Sun uses emissive material with warm glow. Point light emanates from Sun. Planets have subtle rim lighting. Add optional animated surface noise to Sun. Do not overbloom until UI is unreadable.

Acceptance Criteria

- ✓ Sun is visually dominant and warm
- ✓ Bloom is visible but not overwhelming
- ✓ Planet visibility improves with lighting
- ✓ Performance stays at 60 FPS

T25 Saturn Ring & Planet Materials

Polish planet materials: Saturn ring (flat ellipse mesh), Jupiter subtle banding, Earth blue with atmosphere shader, Mars reddish, Venus warm cream, Mercury gray-brown. Use NASA-style free texture maps for Earth if available.

Acceptance Criteria

- ✓ Saturn ring is immediately recognizable
- ✓ Earth is clearly Earth (blue, atmosphere glow)
- ✓ Each planet is visually distinct
- ✓ Overall style is premium 2.5D cosmic

T26 Loading & Error States

Add graceful loading states: 'Calculating birth sky…', 'Preparing solar system…', 'Rewinding time…'. Add error states for: invalid input, timezone failure, astronomy failure, WebGL unsupported. User never sees a frozen blank screen.

Acceptance Criteria

- ✓ Loading states are shown at the right times
- ✓ Error messages are friendly and readable
- ✓ App can recover without page refresh
- ✓ WebGL unsupported shows a clear message

T27 Demo Presets

Add small preset buttons (Kathmandu, Bengaluru, New York, London) that fill latitude, longitude, and timezone fields for quick testing. Keep them subtle — small helper buttons below the main form or in a dev/demo strip.

Acceptance Criteria

- ✓ Presets fill correct lat/long/timezone values
- ✓ User can still edit values manually after filling
- ✓ Presets are visually secondary to the main form

■ Milestone 10 — Final Polish & Handoff

T28 Responsive Layout

Improve layout for tablet and mobile. Tablet: left panel narrows, location card collapsible. Mobile: form becomes bottom sheet, timeline compact, labels hidden by default, control dock simplified.

Acceptance Criteria

- ✓ Desktop looks excellent matching concept image
- ✓ Tablet layout does not break
- ✓ Mobile form is accessible as bottom sheet
- ✓ Canvas remains the main focus on all sizes

T29 Final Visual Polish Pass

Full UI polish pass matching the concept image: glass panel blur/border, button blue glow, orbit ring elegance, timeline glow, status pill polish, planet label subtlety, background depth, spacing, typography.

Acceptance Criteria

- ✓ App feels like a real product, not a prototype
- ✓ Visual design strongly matches concept image
- ✓ All spacing feels intentional
- ✓ Typography feels observatory-grade

T30 README — Usage & Accuracy Note

Update README with: project name, concept, installation, dev/build commands, how birth input works, accuracy disclaimer, visual scaling note, future roadmap. Accuracy note must say: 'Nakshatra calculates date-based planetary positions, but planet sizes and distances are artistically scaled for visual clarity.'

Acceptance Criteria

- ✓ README explains how to run the app
- ✓ README explains the product vision clearly
- ✓ Accuracy limitation is stated honestly
- ✓ New developer can understand the project in 5 minutes

T31 Final MVP Acceptance Test

Run a full manual test with: Date 1992-05-14, Time 07:35, Lat 12.9716, Lon 77.5946, Timezone Asia/Kolkata. Verify all 12 acceptance criteria from the PRD pass. Check for console errors, broken layout, frame drops.

Acceptance Criteria

- ✓ All 12 MVP acceptance criteria pass
- ✓ No console errors
- ✓ No broken layout at any stage
- ✓ No obvious frame drops during rewind
- ✓ Experience feels cinematic and personal

Important: Do not build astrology readings in MVP. Planet sizes and distances are artistically scaled. The user should feel like the universe rewound to the moment they were born.